

ASSIGNMENT NO.5: Aggregate Functions and Data Analysis

Objective:

The objective of this practical is to help students understand how to analyze data using aggregate functions in MySQL. Students will learn how to use COUNT, SUM, AVG, MIN, and MAX, group records using GROUP BY, filter grouped records using HAVING, and analyze trends from datasets.

Introduction to Aggregate Functions

Aggregate functions are used to perform calculations on a group of records and return a single result. These functions are very useful for data analysis and report generation.

For example, in a college database, aggregate functions can be used to find the total number of students, average marks, highest marks, lowest marks, and total fees collected.

Common Aggregate Functions in MySQL

Function	Purpose	Example
COUNT()	Counts number of records	Count total students
SUM()	Calculates total value	Total fees collected
AVG()	Calculates average value	Average marks
MIN()	Finds minimum value	Lowest marks
MAX()	Finds maximum value	Highest marks

Example Use Cases

Requirement	Aggregate Function
Find total students	COUNT()
Find total marks	SUM()
Find average marks	AVG()
Find lowest marks	MIN()
Find highest marks	MAX()

Creating Sample Table and Inserting Records

For this assignment, we will use a table named StudentResult.

StudentResult Table Structure

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	

Course	VARCHAR(30)	
City	VARCHAR(30)	
Marks	INT	
FeesPaid	DECIMAL(10,2)	

MySQL Code to Create Database and Table

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE StudentResult ( StudentID INT PRIMARY KEY, StudentName
VARCHAR(50), Course VARCHAR(30), City VARCHAR(30), Marks INT, FeesPaid
DECIMAL(10,2));
```

MySQL Code to Insert Records

```
INSERT INTO StudentResult
(StudentID, StudentName, Course, City, Marks, FeesPaid)
VALUES
(1, 'Rahul Sharma', 'BSc IT', 'Mumbai', 78, 45000.00),
(2, 'Priya Patil', 'BCA', 'Pune', 85, 40000.00),
(3, 'Aman Verma', 'BSc CS', 'Nashik', 67, 42000.00),
(4, 'Sneha Joshi', 'BSc IT', 'Mumbai', 91, 45000.00),
(5, 'Karan Mehta', 'BCA', 'Thane', 74, 40000.00),
(6, 'Neha Shah', 'BSc IT', 'Pune', 88, 45000.00),
(7, 'Rohan Desai', 'BSc CS', 'Mumbai', 69, 42000.00),
(8, 'Meera Kulkarni', 'BCA', 'Nashik', 82, 40000.00),
(9, 'Arjun Nair', 'BSc IT', 'Thane', 59, 45000.00),
(10, 'Divya More', 'BCA', 'Mumbai', 93, 40000.00);
```

Sample Data

StudentID	StudentName	Course	City	Marks	FeesPaid
1	Rahul Sharma	BSc IT	Mumbai	78	45000.00
2	Priya Patil	BCA	Pune	85	40000.00
3	Aman Verma	BSc CS	Nashik	67	42000.00
4	Sneha Joshi	BSc IT	Mumbai	91	45000.00
5	Karan Mehta	BCA	Thane	74	40000.00
6	Neha Shah	BSc IT	Pune	88	45000.00
7	Rohan Desai	BSc CS	Mumbai	69	42000.00
8	Meera Kulkarni	BCA	Nashik	82	40000.00
9	Arjun Nair	BSc IT	Thane	59	45000.00
10	Divya More	BCA	Mumbai	93	40000.00

Using COUNT, SUM, AVG, MIN, and MAX

Aggregate functions help summarize numerical and record-based data.

Example 1: Count Total Students

```
SELECT COUNT(*) AS TotalStudents FROM StudentResult;
```

Output

TotalStudents
10

Example 2: Find Total Fees Paid

```
SELECT SUM(FeesPaid) AS TotalFeesPaid FROM StudentResult;
```

Output

TotalFeesPaid
424000.00

Example 3: Find Average Marks

```
SELECT AVG(Marks) AS AverageMarks FROM StudentResult;
```

Output

AverageMarks
78.6000

Example 4: Find Lowest and Highest Marks

```
SELECT MIN(Marks) AS LowestMarks, MAX(Marks) AS HighestMarks FROM  
StudentResult;
```

Output

LowestMarks	HighestMarks
59	93

Explanation

Query Part	Meaning
COUNT(*)	Counts all records
SUM(FeesPaid)	Adds all fees paid
AVG(Marks)	Calculates average marks
MIN(Marks)	Finds lowest marks
MAX(Marks)	Finds highest marks
AS	Gives a temporary column name in output

Grouping Data using GROUP BY

The GROUP BY clause is used to group records that have the same value in one or more columns. It is commonly used with aggregate functions.

For example, we can group students by course and find how many students are enrolled in each course.

Syntax

```
SELECT column_name, aggregate_function(column_name) FROM table_name GROUP  
BY column_name;
```

Example 1: Count Students Course-Wise

```
SELECT Course, COUNT(*) AS TotalStudents FROM StudentResult GROUP BY Course;
```

Output

Course	TotalStudents
BSc IT	4
BCA	4
BSc CS	2

Example 2: Find Average Marks Course-Wise

```
SELECT Course, AVG(Marks) AS AverageMarks FROM StudentResult GROUP BY Course;
```

Output

Course	AverageMarks
BSc IT	79.0000
BCA	83.5000
BSc CS	68.0000

Example 3: Find Total Fees Course-Wise

```
SELECT Course, SUM(FeesPaid) AS TotalFees FROM StudentResult GROUP BY Course;
```

Output

Course	TotalFees
BSc IT	180000.00
BCA	160000.00
BSc CS	84000.00

Explanation

SQL Part	Purpose
GROUP BY Course	Groups records course-wise
COUNT(*)	Counts students in each course
AVG(Marks)	Calculates average marks of each course
SUM(FeesPaid)	Calculates total fees paid course-wise

Filtering Grouped Data using HAVING

The HAVING clause is used to filter grouped records. It is similar to WHERE, but WHERE filters individual rows before grouping, while HAVING filters groups after grouping.

Difference Between WHERE and HAVING

Clause	Used For	Used With Aggregate Function
WHERE	Filters individual rows	No
HAVING	Filters grouped data	Yes

Example 1: Display Courses Having More Than 2 Students

```
SELECT Course, COUNT(*) AS TotalStudents FROM StudentResult GROUP BY Course HAVING COUNT(*) > 2;
```

Output

Course	TotalStudents
--------	---------------

BSc IT	4
BCA	4

Example 2: Display Courses Having Average Marks Greater Than 80

```
SELECT Course, AVG(Marks) AS AverageMarks FROM StudentResult GROUP BY Course HAVING AVG(Marks) > 80;
```

Output

Course	AverageMarks
BCA	83.5000

Example 3: Display Cities Having Total Fees Greater Than 80000

```
SELECT City, SUM(FeesPaid) AS TotalFees FROM StudentResult GROUP BY City HAVING SUM(FeesPaid) > 80000;
```

Output

City	TotalFees
Mumbai	172000.00
Pune	85000.00
Nashik	82000.00

Analyzing Trends in Datasets

Aggregate functions, GROUP BY, and HAVING are very useful for finding trends in datasets. Trends help users understand patterns, performance, and business insights.

Trend Analysis Examples

Analysis Requirement	MySQL Query Purpose
Which course has the highest number of students?	Count students course-wise
Which city has more students?	Group students city-wise
Which course has the highest average marks?	Average marks course-wise
Which course collected the highest fees?	Sum fees course-wise
Which course has weak performance?	Find courses with low average marks
Which cities generated more fees?	Group by city and sum fees

Example 1: Analyze Student Count City-Wise

```
SELECT City, COUNT(*) AS TotalStudents FROM StudentResult GROUP BY City ORDER BY TotalStudents DESC;
```

Output

City	TotalStudents
Mumbai	4
Pune	2
Nashik	2
Thane	2

Example 2: Analyze Course Performance

```
SELECT Course, COUNT(*) AS TotalStudents, AVG(Marks) AS AverageMarks,  
MIN(Marks) AS LowestMarks, MAX(Marks) AS HighestMarks FROM StudentResult  
GROUP BY Course;
```

Output

Course	TotalStudents	AverageMarks	LowestMarks	HighestMarks
BSc IT	4	79.0000	59	91
BCA	4	83.5000	74	93
BSc CS	2	68.0000	67	69

Example 3: Display High-Performing Courses

```
SELECT Course, AVG(Marks) AS AverageMarks FROM StudentResult GROUP BY  
Course HAVING AVG(Marks) >= 80 ORDER BY AverageMarks DESC;
```

Output

Course	AverageMarks
BCA	83.5000

Summary Table

Concept	Purpose	Example
COUNT()	Counts records	Total students
SUM()	Adds numeric values	Total fees
AVG()	Calculates average	Average marks
MIN()	Finds lowest value	Lowest marks
MAX()	Finds highest value	Highest marks
GROUP BY	Groups similar records	Course-wise report
HAVING	Filters grouped results	Courses with average marks above 80
ORDER BY	Sorts analysis output	Highest to lowest total students

Questions:

Q1) Create a MySQL database named CollegeDB and a table named StudentResult.

The table should contain the following columns:

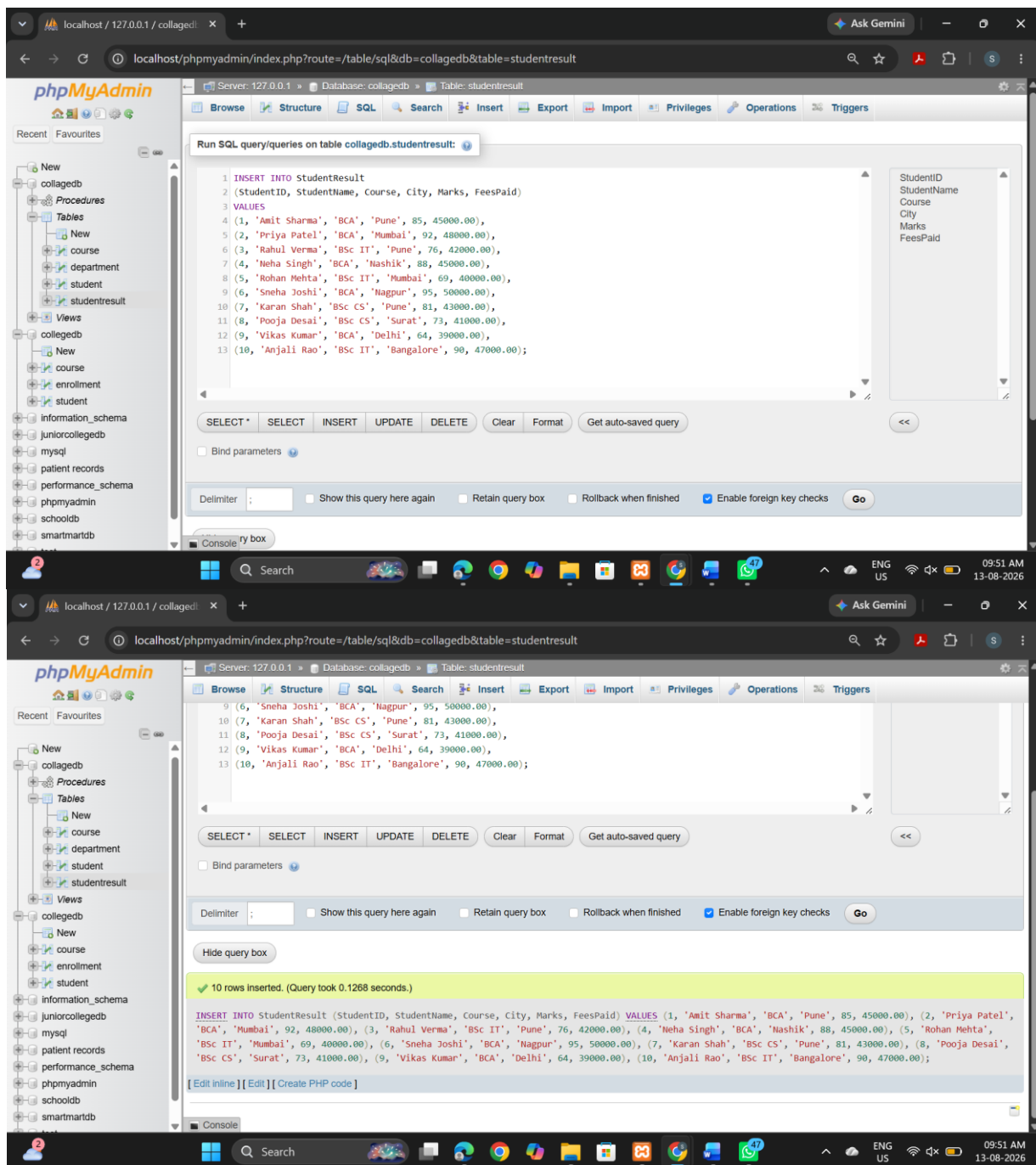
Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	
Course	VARCHAR(30)	
City	VARCHAR(30)	
Marks	INT	
FeesPaid	DECIMAL(10,2)	

Insert at least 10 records into the table.

```
CREATE DATABASE CollegeDB;  
USE CollegeDB;
```

```
CREATE TABLE StudentResult (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50),  
    Course VARCHAR(30),  
    City VARCHAR(30),  
    Marks INT,  
    FeesPaid DECIMAL(10,2)  
);
```

```
INSERT INTO StudentResult (StudentID, StudentName, Course, City, Marks, FeesPaid)  
VALUES  
(1, 'Rahul Sharma', 'BSc IT', 'Mumbai', 78, 45000.00),  
(2, 'Priya Patil', 'BCA', 'Pune', 85, 40000.00),  
(3, 'Aman Verma', 'BSc CS', 'Nashik', 67, 42000.00),  
(4, 'Sneha Joshi', 'BSc IT', 'Mumbai', 91, 45000.00),  
(5, 'Karan Mehta', 'BCA', 'Thane', 74, 40000.00),  
(6, 'Neha Shah', 'BSc IT', 'Pune', 88, 45000.00),  
(7, 'Rohan Desai', 'BSc CS', 'Mumbai', 69, 42000.00),  
(8, 'Meera Kulkarni', 'BCA', 'Nashik', 82, 40000.00),  
(9, 'Arjun Nair', 'BSc IT', 'Thane', 59, 45000.00),  
(10, 'Divya More', 'BCA', 'Mumbai', 93, 40000.00);
```



Q2) Write MySQL queries to find the following from the StudentResult table:

Requirement
Total number of students
Total fees paid
Average marks
Lowest marks
Highest marks

SELECT COUNT(*) AS TotalStudents FROM StudentResult;

SELECT SUM(FeesPaid) AS TotalFeesPaid FROM StudentResult;

SELECT AVG(Marks) AS AverageMarks FROM StudentResult;

SELECT MIN(Marks) AS LowestMarks FROM StudentResult;

SELECT MAX(Marks) AS HighestMarks FROM StudentResult;

The image consists of two screenshots of the phpMyAdmin web interface, showing the execution of SQL queries on a database named 'collegedb'.

Top Screenshot: The 'SQL' tab is active. The query editor contains the following SQL code:

```
1. SELECT COUNT(*) AS TotalStudents
2. FROM studentresult;
```

The table structure for 'studentresult' is displayed on the right:

Field	Type	Nullable	Key
StudentID	INT	NO	PRIMARY
StudentName	VARCHAR(50)	YES	
Course	VARCHAR(50)	YES	
City	VARCHAR(50)	YES	
Marks	INT	YES	
FeesPaid	INT	YES	

Bottom Screenshot: The same SQL query has been executed successfully. A yellow message bar states: "Your SQL query has been executed successfully." Below this, the query is repeated. The result is shown as a single row:

TotalStudents
10

Below the result, there are options for "Query results operations": Print, Copy to clipboard, Export, Display chart, and Create view.

Q3) Write a MySQL query to count the number of students in each course.

The output should contain:

Column
Course
TotalStudents

```
SELECT Course, COUNT(*) AS TotalStudents
FROM StudentResult
GROUP BY Course;
```

The image consists of two screenshots of the phpMyAdmin web interface, demonstrating the execution of a MySQL query.

Top Screenshot: The 'SQL' tab is active. The query editor contains the following SQL statement:

```
1. SELECT Course, COUNT(*) AS TotalStudents
2. FROM studentresult
3. GROUP BY Course;
```

The left sidebar shows the database structure, including the 'collagedb' database and its tables: 'course', 'department', 'student', and 'studentresult'. The right sidebar shows the structure of the 'studentresult' table, including fields like 'StudentID', 'StudentName', 'Course', 'City', 'Marks', and 'FeesPaid'.

Bottom Screenshot: The 'SQL' tab is still active, but the query has been executed. The status bar indicates 'Showing rows 0 - 2 (3 total, Query took 0.0014 seconds.)'. The query text is repeated below the status bar. Below the query, there are options for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. The 'Show all' checkbox is checked, and the 'Number of rows' is set to 25. The 'Filter rows' field is empty. The 'Extra options' section is expanded, showing the results of the query in a table format:

Course	TotalStudents
BCA	5
BSc CS	2
BSc IT	3

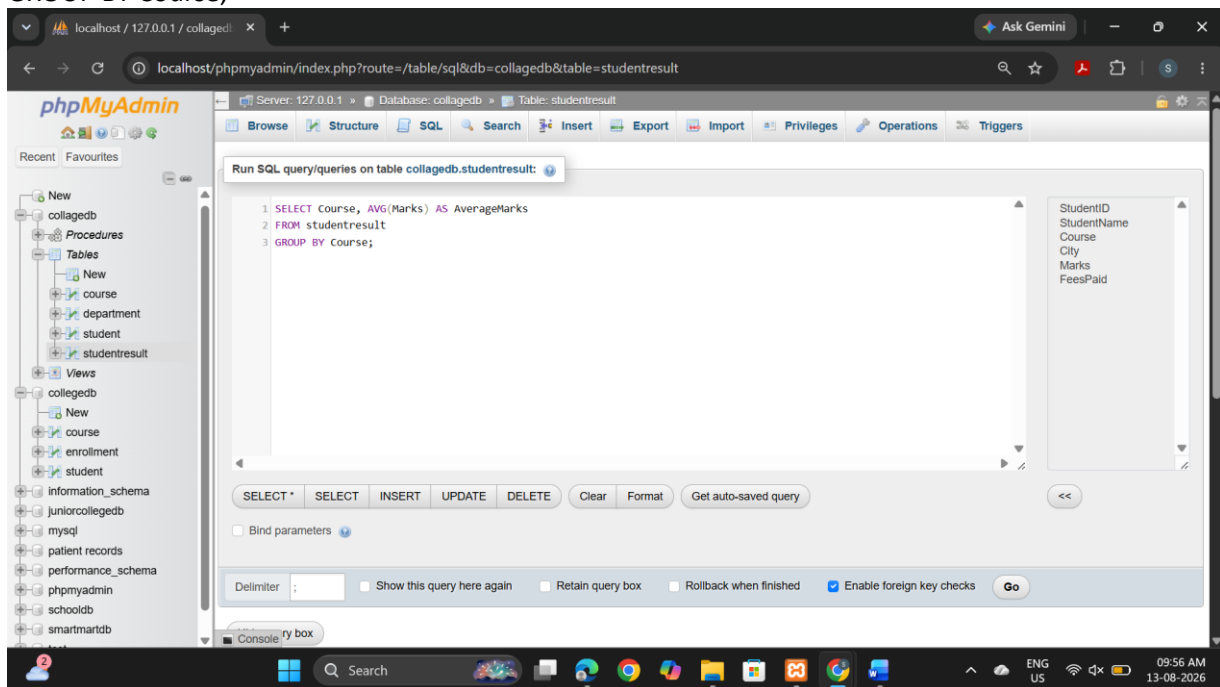
Below the table, there are options for 'Show all', 'Number of rows' (set to 25), and 'Filter rows'. The 'Query results operations' section is expanded, showing options for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

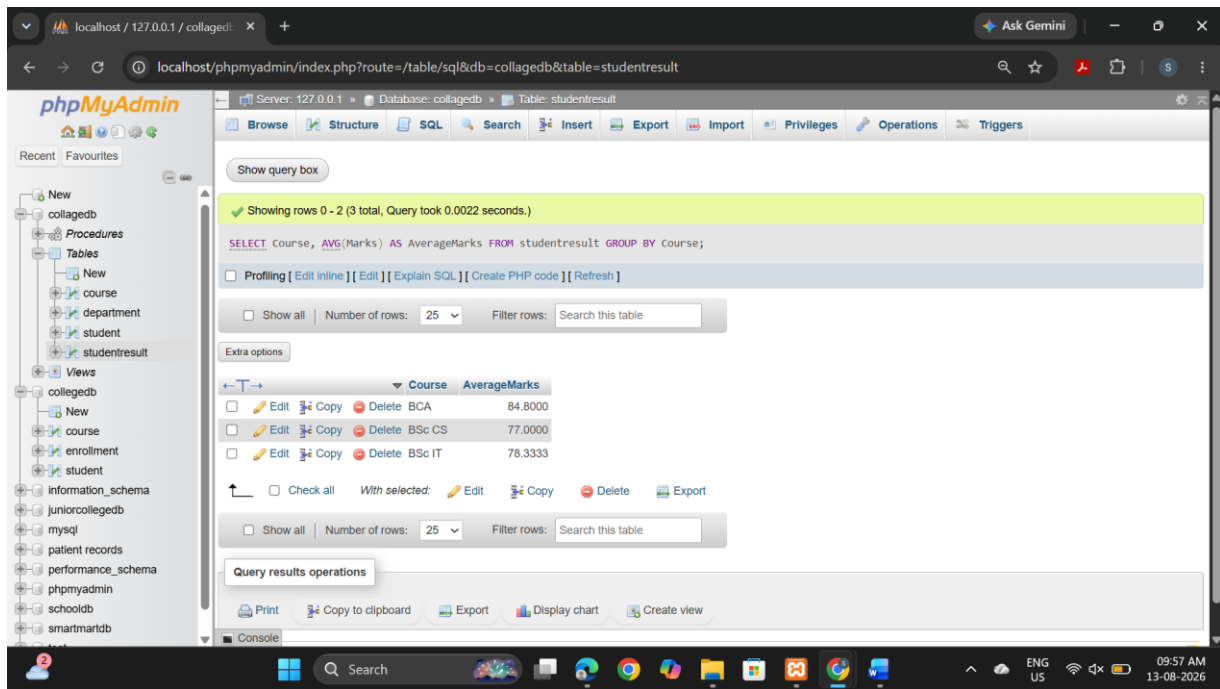
Q4) Write a MySQL query to find the average marks of students course-wise.

The output should contain:

Column
Course
AverageMarks

```
SELECT Course, AVG(Marks) AS AverageMarks
FROM StudentResult
GROUP BY Course;
```





Q5) Write a MySQL query to display only those courses where the number of students is greater than 2.

Use:

Clause

GROUP BY

HAVING

SELECT Course, COUNT(*) AS TotalStudents
FROM StudentResult
GROUP BY Course

HAVING COUNT(*) > 2;

The image consists of two screenshots of the phpMyAdmin web interface. The top screenshot shows the SQL query editor with the following query:

```
1. SELECT Course, Count(*) AS TotalStudents
2. FROM studentresult
3. GROUP BY Course
4. HAVING COUNT(*) > 2;
```

 The bottom screenshot shows the results of the query, displaying a table with two rows:

Course	TotalStudents
BCA	5
BSc IT	3

Q6) Write a MySQL query to analyze course performance.

The output should display:

Column
Course
Column
TotalStudents
AverageMarks

LowestMarks

HighestMarks

Sort the result by AverageMarks in descending order.

```
SELECT Course,  
       COUNT(*) AS TotalStudents,  
       AVG(Marks) AS AverageMarks,  
       MIN(Marks) AS LowestMarks,  
       MAX(Marks) AS HighestMarks  
FROM StudentResult  
GROUP BY Course  
ORDER BY AverageMarks DESC;
```

The screenshot shows the phpMyAdmin interface with the SQL query executed successfully. The query is: `SELECT Course, COUNT(*) AS TotalStudents, AVG(Marks) AS AverageMarks, MIN(Marks) AS lowestMarks, MAX(Marks) AS HighestMarks FROM studentresult GROUP BY Course ORDER BY AverageMarks DESC;`

The result shows 3 rows of data:

Course	TotalStudents	AverageMarks	lowestMarks	HighestMarks
BCA	5	84.8000	64	95
BSc IT	3	78.3333	69	90
BSc CS	2	77.0000	73	81

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well Done []

Signature of Instructor: _____

